BA305 — Exam 1 · Version 1 · Solutions

Total: 66 points. Part A: 41 points · Part B: 25 points.

Part A — Concepts (Questions 1–15, 41 points)

No code in this part.

Part A: 41 points.

Question 1 — True / False (3 points)

A customer table has a column `zip_code` with values such as 10001. Before running k-NN, you should standardize the `zip_code` column.

- True
- False

> **Answer — False.** A ZIP code is a label, not a quantity: the gap between two ZIP codes says nothing about how similar two customers are. Treat it as a category (dummies or regions), or drop it.

Question 2 — Multiple choice (2 points)

A churn dataset has 2,000 customers. The column `age` is missing for 800 of them; every other column is complete. The team expects age to matter for predicting churn. What is the **most** sensible way to handle the missing ages?

- (a) Delete every customer that has a missing value.
- (b) Fill each missing age with the average age of the other customers.
- (c) Delete the `age` column.
- (d) Fill each missing age with 0.

> **Answer — (b).** Filling with the mean keeps all 2,000 customers and the age column. (a) loses 40% of the data, (c) loses a useful feature, (d) makes 800 customers look 0 years old. The median is also acceptable.

Question 3 — Multiple choice (3 points)

A team standardizes every feature of its customer data before running k-NN. Which statement is true after standardizing?

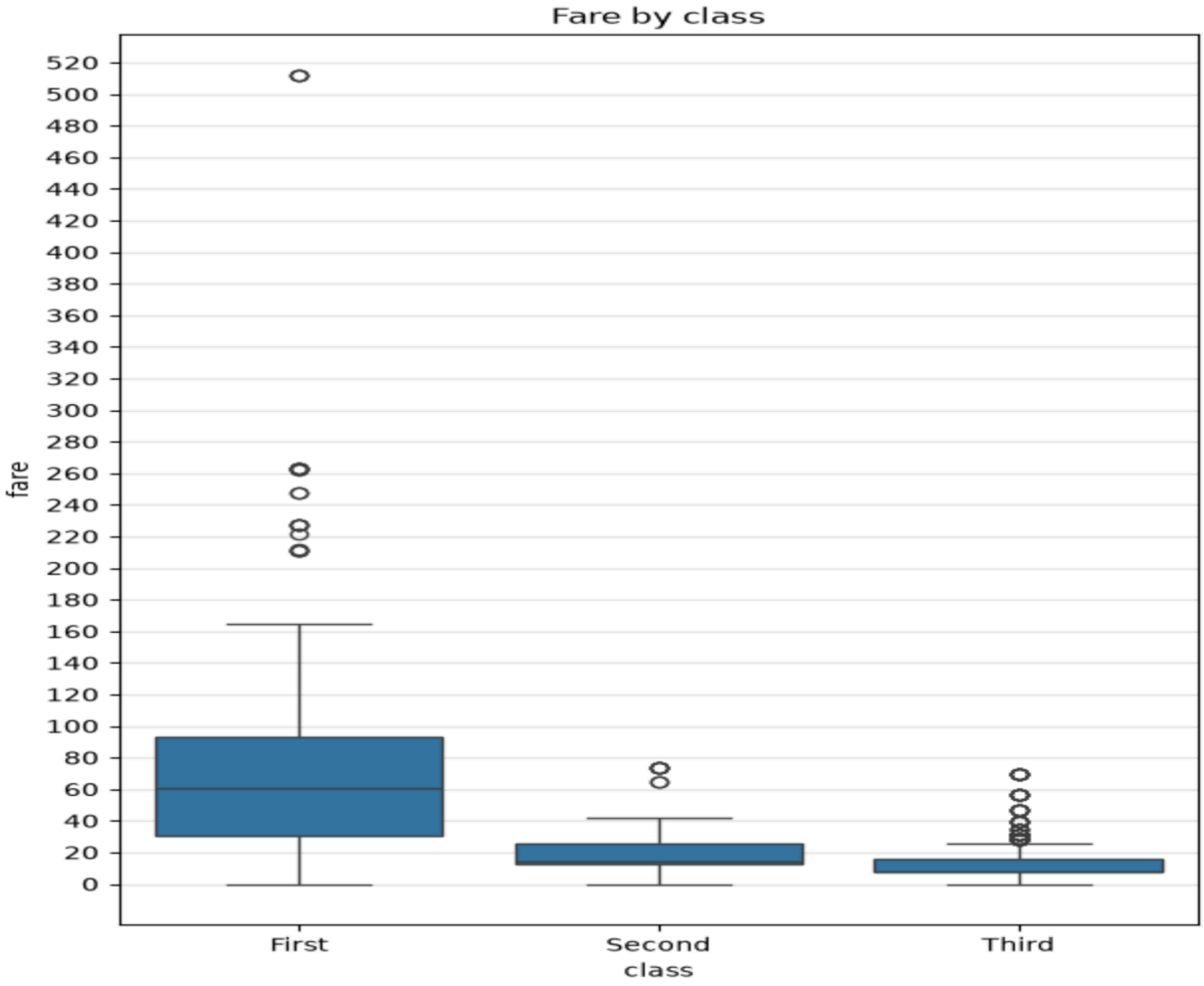
- (a) Every feature has a mean of 0 and a standard deviation of 1.
- (b) Every row has a mean of 0 and a standard deviation of 1.
- (c) Every row's values lie between 0 and 1.

- (d) None of the other options.

> **Answer — (a).** Standardizing works one feature (column) at a time, giving each mean 0 and standard deviation 1. (b) mixes up rows and columns; (c) describes min-max scaling.

Question 4 — Select all that apply (4 points)

The boxplot shows the fare (in pounds) paid by the 891 Titanic passengers, by ticket class.



Select **all** statements this boxplot supports.

- (a) The median first-class fare is about 60.
- (b) At least 75% of third-class passengers paid less than 20.
- (c) No second-class passenger paid more than about 42.
- (d) Some first-class passengers paid nothing.
- (e) The mean first-class fare is about 60.

> **Answer — (a), (b) and (d).** (a) The first-class median line is at about 60. (b) The third-class 75th percentile is about 15. (c) False: outliers sit above the whisker. (d) The first-class whisker reaches 0. (e) False: a boxplot

> shows the median, not the mean (the mean is 84).

Question 5 — Multiple choice (2 points)

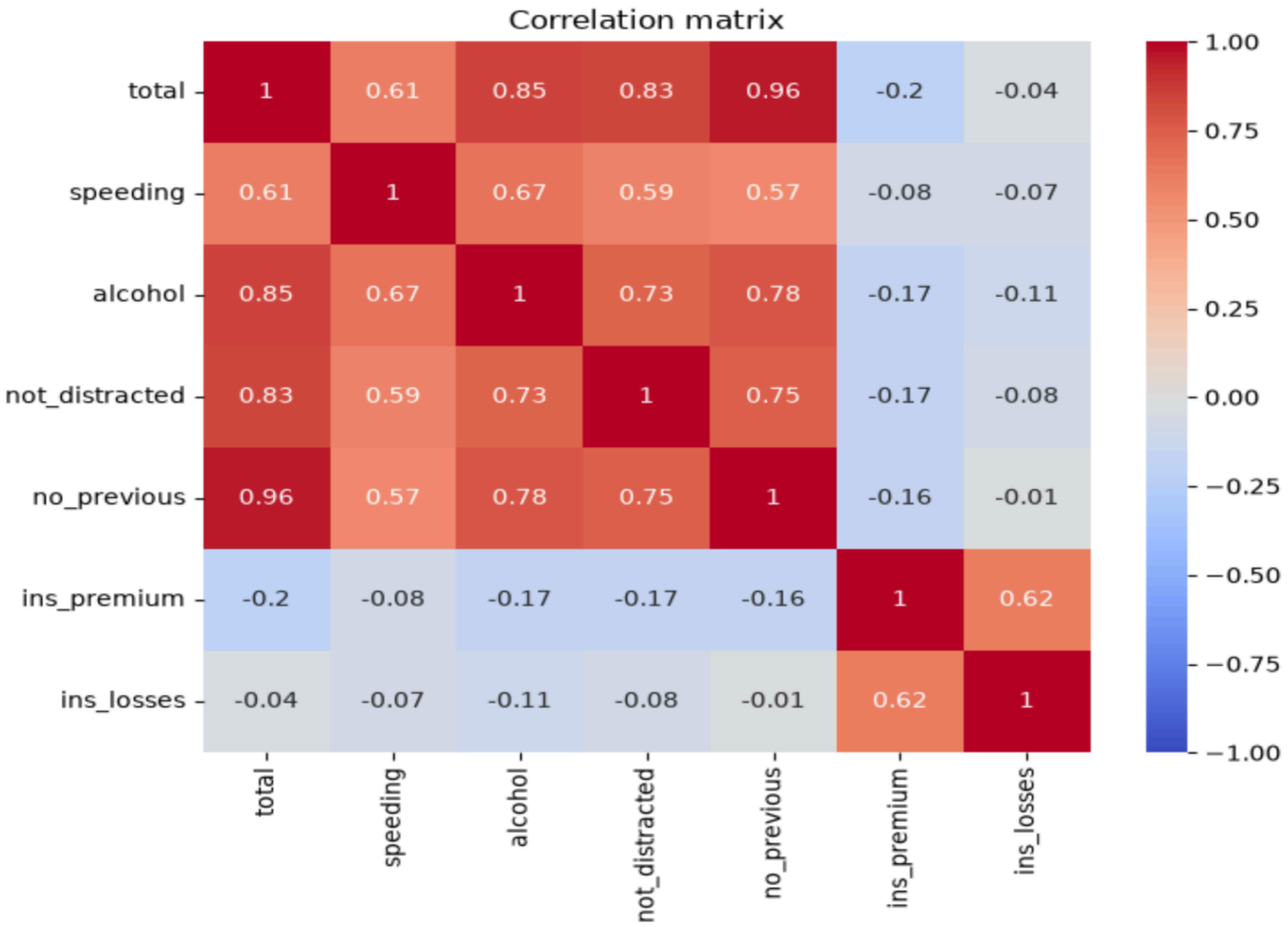
A manager wants to see how the average order value changed month by month over the last three years. Which chart fits best?

- (a) A pie chart with one slice per month
- (b) A line chart with months on the x-axis
- (c) A histogram of the 36 monthly averages
- (d) A scatterplot of order value against customer age

> **Answer — (b).** A line chart is the chart for a trend over time. (a) cannot compare 36 slices, (c) loses the time order, (d) answers a different question.

Question 6 — Multiple choice (2 points)

The heatmap shows the correlations between seven measures of US car crashes and car insurance. The data has one row per US state. Five columns are rates of drivers in fatal crashes per billion miles driven: `total`, and those who were `speeding`, `alcohol`-involved, `not_distracted`, or had `no_previous` accidents. `ins_premium` is the average yearly car-insurance premium ($) and `ins_losses` the insurers' yearly losses per insured driver ($).



Which pair of **different** columns is most strongly correlated?

- (a) The total fatal-crash rate and the insurance premium
- (b) The total fatal-crash rate and the rate for drivers with no previous accidents
- (c) The insurance premium and the insurers' losses
- (d) The speeding rate and the alcohol-involved rate

> **Answer — (b), 0.96.** Strength is the size of the correlation: (a) is only −0.20, (c) 0.62, (d) 0.67.

Question 7 — Multiple choice (3 points)

An analyst has seven measures of US car crashes and car insurance. The data has one row per US state. Five columns are rates of drivers in fatal crashes per billion miles driven: `total`, and those who were `speeding`, `alcohol`-involved, `not_distracted`, or had `no_previous` accidents. `ins_premium` is the average yearly car-insurance premium ($) and `ins_losses` the insurers' yearly losses per insured driver ($). Their summary statistics:

	total	speeding	alcohol	not_distracted	no_previous	ins_premium	ins_losses
count	51.0	51.0	51.0	51.0	51.0	51.0	51.0
mean	15.8	5.0	4.9	13.6	14.0	887.0	134.5
std	4.1	2.0	1.7	4.5	3.8	178.3	24.8
min	5.9	1.8	1.6	1.8	5.9	642.0	82.8
25%	12.8	3.8	3.9	10.5	11.3	768.4	114.6
50%	15.6	4.6	4.6	13.9	13.8	859.0	136.0
75%	18.5	6.4	5.6	16.1	16.8	1007.9	151.9
max	23.9	9.4	10.0	23.7	21.3	1301.5	194.8

The analyst runs PCA on the seven columns as they are, without standardizing them first. The first component alone explains 98.7% of the variance. What is the best explanation?

- (a) All seven measures move together so closely that one component sums them all up.
- (b) The insurance premium is in dollars and varies by hundreds, far more than the other columns, so it dominates.
- (c) PCA always gives most of the variance to the first component, whatever the data.
- (d) 51 states are too few rows for PCA to give a sensible result.

> **Answer — (b).** PCA follows variance, and variance depends on units. `ins_premium` (SD ≈ 178) dwarfs the crash columns (SD ≈ 2–4), so PC1 is essentially that one column. After standardizing, PC1 explains 57%.

Question 8 — True / False (2 points)

When you run PCA on a set of features before predicting churn, PCA picks the components that best separate churners from non-churners.

- True
- False

> **Answer — False.** PCA is unsupervised: it never sees the target. It finds the directions of most variance, which may or may not separate churners.

Question 9 — True / False (3 points)

A bank wants to predict, before calling, which customers will accept an offer. Its data includes `call_duration`, the length of the sales call. Including `call_duration` raises test accuracy a lot, so it should be kept as a feature.

- True
- False

> **Answer — False.** Call length is only known after the call, so it is not available when the prediction is made. This is leakage.

Question 10 — Select all that apply (4 points)

A spam filter is tested on 1,000 emails, 100 of which really are spam. It flags 120 emails as spam, and 80 of those really are spam. Treat spam as the positive class. Select **all** that are true.

- (a) Accuracy = 940 / 1000
- (b) Precision = 80 / 100
- (c) Recall = 80 / 100
- (d) False positive rate = 40 / 900
- (e) The filter missed 40 spam emails.

> **Answer — (a), (c) and (d).** TP = 80, FP = 40, FN = 20, TN = 860. Accuracy = 940/1000, recall = 80/100, FPR = 40/900. (b) Precision is 80/120. (e) It missed 20 spam emails, not 40.

Question 11 — True / False (3 points)

A model's R^2 on the test set is −0.2. This can happen: it means the model predicts worse than simply guessing the average for every record.

- True
- False

> **Answer — True.** $R^2 = 1 - SSE/SST$ is negative when the model does worse than predicting the average (SSE > SST), which can happen on test data.

Question 12 — Multiple choice (2 points)

During pre-processing, what is the primary purpose of applying the hold-out method to the data?

- (a) It cleans the data, e.g., by removing missing values.
- (b) It makes the dataset easier to work with by separating it into smaller, more manageable chunks.
- (c) It gives us a better idea of the out-of-sample performance of a model.
- (d) It gives us a better idea of the in-sample performance of a model, but it also likely increases overfitting risk.
- (e) It gives us a better idea of the in-sample performance of a model, but it also likely decreases overfitting risk.

> **Answer — (c).** A held-out test set measures performance on new (out-of-sample) data.

Question 13 — Multiple choice (3 points)

A model scores each customer's probability of churning. You lower the cutoff for "will churn" from 0.5 to 0.3. Which is most likely?

- (a) Recall goes up or stays the same; precision usually goes down.
- (b) Recall goes down; precision goes up.
- (c) Both recall and precision go up.
- (d) Nothing changes, because the model is the same.

> **Answer — (a).** A lower cutoff flags more customers: recall can only rise or stay the same, while the extra flags include more false alarms, so precision usually drops.

Question 14 — Multiple choice (2 points)

A new customer's five nearest training neighbors, from closest to farthest, are:

Neighbor	Distance	Bought?
1	0.5	Yes
2	0.8	Yes
3	1.1	No
4	1.4	No
5	1.5	No

What does k-NN predict with k = 3 and with k = 5?

- (a) Yes and Yes
- (b) Yes and No
- (c) No and No
- (d) No and Yes

> **Answer — (b).** k = 3: Yes, Yes, No → Yes. k = 5: Yes, Yes, No, No, No → No.

Question 15 — True / False (3 points)

A 1-nearest-neighbor model scores 100% accuracy on its own training data. This shows the model will predict new customers well.

- True

- False

> **Answer — False.** With k = 1 each training point is its own nearest neighbor, so training accuracy is near 100% for any data. Only test or validation data shows how it does on new customers.

Part B — Coding questions (Questions 16–23, 25 points)

The code and its output are in two separate handouts, **Handout B1** and **Handout B2**. Each question names the exhibits it uses. You cannot run the code.

Part B: 25 points. Total: 66 points.

Case B1 — Online Shoppers (use Handout B1)

An online store logged 12,330 visits (sessions). For each one it recorded how many pages of each type the visitor viewed and for how long, Google-Analytics measures such as `PageValues` and `BounceRates`, the month, codes for the visitor's operating system, browser, region and traffic source, the visitor type, and whether the session ended in a purchase (`Revenue`).

Exhibit 1

```
# Load the shopper sessions and look at the share that ended in a purchase.
shoppers = pd.read_csv(url)
print('Sessions:', len(shoppers))
shoppers['Revenue'].value_counts(normalize=True).round(3)
```

```
Sessions: 12330
Revenue
False    0.845
True     0.155
Name: proportion, dtype: float64
```

Exhibit 2

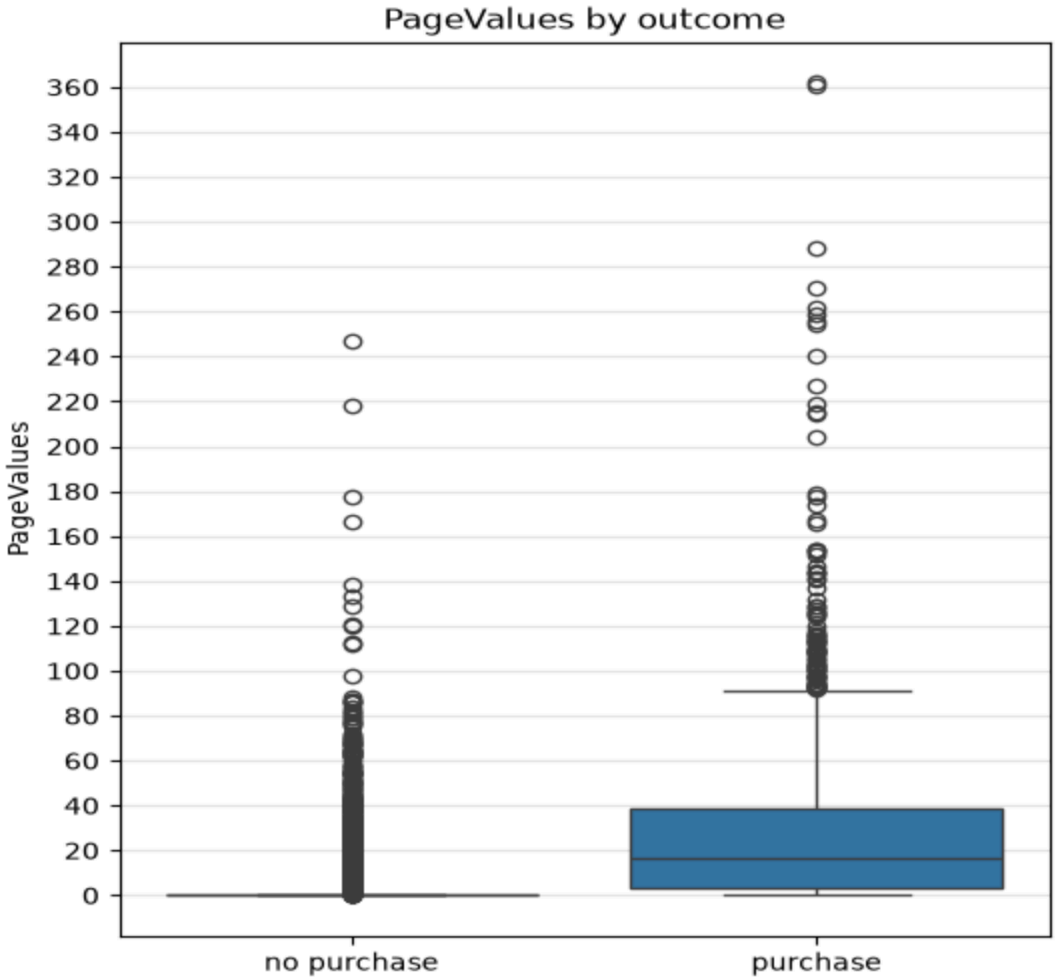
```
# A few of the columns.
shoppers[['ProductRelated', 'PageValues', 'Month', 'Region', 'VisitorType', 'Weekend',
'Revenue']].head()
```

	ProductRelated	PageValues	Month	Region	VisitorType	Weekend	Revenue
0	1	0.0	Feb	1	Returning_Visitor	False	False
1	2	0.0	Feb	1	Returning_Visitor	False	False
2	1	0.0	Feb	9	Returning_Visitor	False	False
3	2	0.0	Feb	2	Returning_Visitor	False	False
4	10	0.0	Feb	1	Returning_Visitor	True	False

Question 16 — Multiple choice (2 points)

Exhibit 3

```
# PageValues for sessions that did and did not end in a purchase.
plt.figure(figsize=(6, 7))
sns.boxplot(data=shoppers, x='Revenue', y='PageValues', order=[False, True])
plt.xticks([0, 1], ['no purchase', 'purchase'])
plt.yticks(range(0, 380, 20))
plt.grid(axis='y', alpha=0.4)
plt.xlabel('')
plt.title('PageValues by outcome');
```



Which statement does this boxplot support?

- (a) At least 75% of the sessions that did not end in a purchase have a `PageValues` of 0.
- (b) At least half of the sessions that ended in a purchase have a `PageValues` of 0.
- (c) All sessions that did not end in a purchase have a `PageValues` below 100.
- (d) Both groups have about the same median `PageValues`.

> **Answer — (a).** The "no purchase" box is a flat line at 0, so its 75th percentile is 0. (b) The purchase median is about 17. (c) Some outliers exceed 100. (d) The medians are 0 and 17.

Question 17 — Multiple choice (3 points)

Exhibit 4

```
# Features: every column except Revenue, text columns turned into 0/1 columns.
# Target: 1 if the session ended in a purchase.
X = pd.get_dummies(shoppers.drop(columns='Revenue'), dtype=int)
y = shoppers['Revenue'].astype(int)

# Hold out 30% for testing, then scale with the training part only.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)
```

Exhibit 5

```
# Training and test accuracy of k-NN for a few values of k.
rows = []
for k in [1, 5, 15, 51]:
    knn = KNeighborsClassifier(n_neighbors=k).fit(X_train_s, y_train)
    rows.append({'k': k,
                 'train accuracy': round(accuracy_score(y_train, knn.predict(X_train_s)), 3),
                 'test accuracy': round(accuracy_score(y_test, knn.predict(X_test_s)), 3)})
pd.DataFrame(rows)
```

	k	train accuracy	test accuracy
0	1	1.000	0.845
1	5	0.900	0.877
2	15	0.882	0.876
3	51	0.865	0.867

A colleague says: "k = 1 is clearly the best model; it is 100% accurate." Which response is best?

- (a) Agree: k = 1 fits every training session exactly, so it has learned the buying patterns best.
- (b) Agree: no other k reaches a higher accuracy anywhere in the table.
- (c) Disagree: on new sessions, k = 1 does no better than always predicting "no purchase".
- (d) Disagree: k = 51 is best, because its training and test accuracies are closest together.

> **Answer — (c).** The 100% is training accuracy. On the test set k = 1 scores 0.845, exactly the naïve rule (84.5% don't buy). (d) A small train–test gap is not the goal: k = 5 has the higher test accuracy.

Question 18 — Multiple choice (3 points)

Exhibit 6

```
# Logistic regression gives each test session a probability of buying.
logit = LogisticRegression(max_iter=2000).fit(X_train_s, y_train)
prob = logit.predict_proba(X_test_s)[:, 1]

# Number of errors, accuracy, precision and recall at several thresholds.
rows = []
for t in [0.1, 0.2, 0.3, 0.4, 0.5, 0.7]:
    pred = (prob >= t).astype(int)
    rows.append({'threshold': t,
                 'flagged as buyers': pred.sum(),
                 'errors': (pred != y_test).sum(),
                 'accuracy': round(accuracy_score(y_test, pred), 3),
                 'precision': round(precision_score(y_test, pred), 3),
                 'recall': round(recall_score(y_test, pred), 3)})
pd.DataFrame(rows)
```

	threshold	flagged as buyers	errors	accuracy	precision	recall
0	0.1	1497	1037	0.720	0.345	0.902
1	0.2	738	504	0.864	0.546	0.705
2	0.3	451	377	0.898	0.716	0.565
3	0.4	360	394	0.893	0.747	0.470
4	0.5	295	411	0.889	0.773	0.399
5	0.7	208	452	0.878	0.788	0.287

The store will open a live-chat helper for every session flagged as a likely buyer. **Chat agents are expensive**, but the store wants to reach **at least 70% of real buyers**. Which threshold should it use?

- (a) 0.1
- (b) 0.2
- (c) 0.3
- (d) 0.5

> **Answer — (b) 0.2.** Only 0.1 and 0.2 reach recall ≥ 0.70, and 0.2 flags half as many sessions (fewer agents) with higher precision. (c) has the highest accuracy but misses the recall target.

Question 19 — Short answer (4 points)

Look again at the threshold table in Exhibit 6. A colleague skips the probabilities and classifies the test sessions directly:

Exhibit 7

```
pred = logit.predict(X_test_s)
```

How many errors will this model make on the test sessions? Explain. Answer in at most 2 sentences.

> **Model answer — 411.** `predict` uses the default 0.5 threshold, so its errors are the 0.5 row of the table.
>
> **Rubric (4 pts):** 4 = 411 with the 0.5-threshold reason · 2 = 411 with no or a wrong reason · 0 = any other number (377 is the best threshold, which `predict` does not search for).

Case B2 — NYC Taxi Rides (use Handout B2)

6,433 yellow and green taxi rides in New York City from March 2019, with the distance (miles), the fare, the tip, tolls, the total charged, the payment type and the pickup and drop-off zones. A taxi app wants to **predict the tip** (in dollars).

- `total` is the full amount the rider paid: **total = fare + tip + tolls + fixed taxes and surcharges**.
- Tips paid in cash are not recorded (they show as $0), so the models below use the **credit-card rides only**.

Exhibit 1

```
# Load the taxi rides and drop the rides with a missing value.
taxis = pd.read_csv(url)
taxis = taxis.dropna().reset_index(drop=True)
print('Rides:', len(taxis))
taxis[['distance', 'fare', 'tip', 'tolls', 'total', 'payment', 'pickup_borough']].head()
```

```
Rides: 6341
```

	distance	fare	tip	tolls	total	payment	pickup_borough
0	1.60	7.0	2.15	0.0	12.95	credit card	Manhattan
1	0.79	5.0	0.00	0.0	9.30	cash	Manhattan
2	1.37	7.5	2.36	0.0	14.16	credit card	Manhattan
3	7.70	27.0	6.15	0.0	36.95	credit card	Manhattan
4	2.16	9.0	1.10	0.0	13.40	credit card	Manhattan

Question 20 — Multiple choice (3 points)

Exhibit 2

```
# Keep the credit-card rides only.
cards = taxis[taxis['payment'] == 'credit card']
```

Exhibit 3

```
# Linear regression on credit-card rides: predict the tip from four columns.
features = ['distance', 'fare', 'tolls', 'total']
X_train, X_test, y_train, y_test = train_test_split(cards[features], cards['tip'],
                                                    test_size=0.3, random_state=1)
model = LinearRegression().fit(X_train, y_train)
print('Test R²:', round(r2_score(y_test, model.predict(X_test)), 3))
```

```
Test R²: 0.903
```

The test R^2 is 0.903. What is the most likely explanation?

- (a) The model is underfitting: it is too simple to capture how tips really vary.
- (b) The model is overfitting: it memorized the training rides and will fail on new ones.
- (c) The model is cheating: it uses a value that is not known at the time of prediction.
- (d) The model's results are unreliable, because the features were not put on the same scale.

> **Answer — (c).** `total` already contains the tip, so the model nearly subtracts its way to the answer, and `total` is not known when the ride is booked. This is leakage. Without it, R^2 drops to 0.283.

Question 21 — Select all that apply (3 points)

Exhibit 4

```
# Same model with 'passengers' in place of 'total'.
features = ['distance', 'fare', 'tolls', 'passengers']
X_train, X_test, y_train, y_test = train_test_split(cards[features], cards['tip'],
                                                    test_size=0.3, random_state=1)
model = LinearRegression().fit(X_train, y_train)
pred = model.predict(X_test)

# Baseline: predict every ride's tip as the average training tip.
baseline = np.full(len(y_test), y_train.mean())
print('Test R²:     ', round(r2_score(y_test, pred), 3))
print('Model MAE:   ', round(mean_absolute_error(y_test, pred), 2))
print('Baseline MAE:', round(mean_absolute_error(y_test, baseline), 2))
```

```
Test R²:      0.283
Model MAE:    1.18
Baseline MAE: 1.52
```

Select **all** that are true.

- (a) On a typical ride, the model's prediction is off by about $1.18.
- (b) R^2 is less than 0.5, so the model does worse than random guessing.
- (c) The model gets about 28% of the tips exactly right.
- (d) Because R^2 is above 0, the model predicts tips better than always guessing the average tip.

> **Answer — (a) and (d).** MAE is in dollars: off by about $1.18 on a typical ride. Any R^2 above 0 beats always guessing the average. (b) The benchmark is 0, not 0.5. (c) R^2 is not a share of exact hits.

Question 22 — Multiple choice (3 points)

Exhibit 5

```
# k-NN regression on the scaled features: mean absolute error (in dollars) for several k.
scaler = StandardScaler().fit(X_train)
rows = []
for k in [1, 3, 5, 10, 20, 50, 100]:
    knn = KNeighborsRegressor(n_neighbors=k).fit(scaler.transform(X_train), y_train)
    rows.append({'k': k,
                 'train MAE': round(mean_absolute_error(y_train,
knn.predict(scaler.transform(X_train))), 2),
                 'test MAE': round(mean_absolute_error(y_test, knn.predict(scaler.transform(X_test))),
2)})
pd.DataFrame(rows)
```

	k	train MAE	test MAE
0	1	0.19	1.41
1	3	0.83	1.24
2	5	0.92	1.19
3	10	1.00	1.14
4	20	1.06	1.15
5	50	1.12	1.15
6	100	1.16	1.17

Which k should you choose?

- (a) k = 1, because its training error is the lowest.
- (b) k = 10, because its test error is the lowest.
- (c) k = 100, because its training and test errors are the closest.
- (d) It cannot be decided from this table, because the test errors are all within $0.30 of each other.

> **Answer — (b) 10.** Choose k by the lowest test (validation) error: $1.14 at k = 10. (a) k = 1 overfits. (c) A small gap is not the goal. (d) Close values still rank.

Question 23 — Short answer (4 points)

You rerun the k-NN comparison of Exhibit 5, changing only one setting in the split:

Exhibit 6

```
X_train, X_test, y_train, y_test = train_test_split(cards[features], cards['tip'],
                                                    test_size=0.9, random_state=1)
```

What do you expect will happen to the test MAE, and why? Answer in at most 3 sentences.

> **Model answer — worse (higher MAE).** Training on only 10% of the rides leaves fewer, less similar neighbors, so predictions get worse (rerun: $1.17 vs. $1.14 at k = 10).

Rubric (4 pts): 4 = worse, because there is much less training data · 2 = worse, with no or a wrong reason · 0 = better or no change.